

FROM GAMER TO GAME DEV

Free Game Dev Roadmap: start at zero, learn the code path, and build toward a job portfolio or indie game.

GETTING STARTED: FIRST STEPS

NO DEGREE NEEDED

You do need a real plan and steady practice.

1 FIRST PLAYABLE

THE PROBLEM

You are starting from zero. You want to go from playing games to building a tiny game loop yourself.

THE NEXT STEP

Beginner proof: one clear player action, one goal, one fail state, one restart, and visible feedback. The smallest loop that feels like a game.

2 PROGRAMMING FUNDAMENTALS

THE PROBLEM

Tutorials can show you what to type, but they do not teach why the code works or how to change it safely.

THE NEXT STEP

Beginner code skills: variables, choices, loops, collections, functions, objects, input, and debugging. Code that changes game rules on purpose.

3 MECHANIC BUILDER

THE PROBLEM

Features still feel hard when you try to build them without copying a guide step by step.

THE NEXT STEP

Intermediate mechanics: pathfinding, spells or abilities, dialogue, crafting, quests, inventory rules, enemy states, and save data. One system at a time.

4 FIRST PROJECT CHOICE

THE PROBLEM

Once the basics work, it is easy to chase too many ideas and never finish the next step.

THE NEXT STEP

A direction comes next. Job path: proof that the code is clean and explainable. Indie path: proof that the game idea can hold a player.

FOUNDATIONS BUILT: CHOOSE A PATH

CHOOSE YOUR OUTCOME

JOB PORTFOLIO PATH

Show studios the technical skills behind your code.

5A SYSTEMS PRACTICE

THE PROBLEM

Screenshots do not prove technical skill. Employers need to see systems, logic, and debugging.

THE NEXT STEP

Intermediate tech proof: gameplay systems with clear rules, edge cases, data flow, debug notes, and readable code structure.

6A TECHNICAL GAME SLICE

THE PROBLEM

Loose experiments do not show whether your code can support a complete feature without falling apart.

THE NEXT STEP

Advanced tech proof: connected systems, clean data ownership, error handling, save/load flow, debug tools, and clear implementation notes.

7A TECHNICAL BREAKDOWN

THE PROBLEM

Good code gets ignored when the technical decisions are hard to inspect or explain.

THE NEXT STEP

Portfolio proof: clean project files, setup notes, system diagrams, bug notes, code comments where needed, and a plain explanation of tradeoffs.

8A APPLY-READY

THE PROBLEM

A job portfolio needs proof of technical judgment, not a larger project or more content.

THE NEXT STEP

Apply-ready proof: focused technical projects, skill-based resume bullets, architecture notes, and answers for bugs, tradeoffs, and scope.

INDIE GAME PATH

Turn small mechanics into one finishable game demo.

5B CORE LOOP PROTOTYPE

THE PROBLEM

Most first game ideas start with story, art, and features before the loop is fun.

THE NEXT STEP

Intermediate indie proof: a core loop with player action, risk, reward, restart flow, and a reason to try again. Genre can change; the loop must work.

6B VERTICAL SLICE

THE PROBLEM

The loop works, but the game may still feel rough, slow, or unclear.

THE NEXT STEP

Advanced indie proof: one polished section that shows feel, feedback, pacing, readability, and the promise of the full game.

7B PLAYTEST DEMO

THE PROBLEM

Playing alone hides bad controls, unclear goals, slow restarts, and boring parts.

THE NEXT STEP

Public proof: a playable demo page, clear instructions, feedback questions, and notes on where players stop or get confused.

8B INDIE-GAME READY

THE PROBLEM

Without a lock, version 1.0 keeps growing.

THE NEXT STEP

Release proof: locked scope, must-have features, cut list, bug list, playtest plan, store page tasks, and a clear finish line.